NPTEL Deep Learning (noc26_cs66) — IIT Ropar

Comprehensive Exam Revision & Case-Study Master Guide

Contents

1 Core Optimization Algorithms & Contours **3**

1.1 The Error Landscape & Gradient Descent (GD) . . . 3

1.2 Momentum-Based Gradient Descent . . . 3

1.3 Nesterov Accelerated Gradient (NAG) . . . 3

1.4 AdaGrad (Adaptive Gradient Algorithm) . . . 4

1.5 RMSProp . . . 4

1.6 Adam (Adaptive Moment Estimation) . . . 4

2 Weight Initialization, Saturation & Batch Normalization **5**

2.1 Pathologies of Naive Initialization . . . 5

2.2 Principled Initialization Schemes . . . 5

2.3 Activation Functions Comparison . . . 6

2.4 Batch Normalization (BN) . . . 6

3 Linear Algebra Foundations: EVD, SVD & PCA **6**

3.1 Eigenvalue Decomposition (EVD) . . . 6

3.2 Singular Value Decomposition (SVD) . . . 7

3.3 Principal Component Analysis (PCA) . . . 7

4 Autoencoders, Weight Tying & Regularization **7**

4.1 Autoencoder Formulations . . . 7

4.2 Linear Autoencoders $\iff$ PCA Equivalence . . . 7

4.3 Structural & Functional Regularizations . . . 8

5 Bias-Variance Tradeoff, True Error & Ensembling **8**

5.1 Bias-Variance Decomposition . . . 8

5.2 True Error vs. Empirical Training Error . . . 8

5.3 Regularization Connections & Equivalences . . . 9

5.4 Ensemble Error Formula & Dropout . . . 9

6 Word Representations, Word2Vec, GloVe & SVD Equivalence **9**

6.1 Vector Representations . . . 9

6.2 Word2Vec Framework . . . 9

6.3 The Grand Connection: Word2Vec $\iff$ SVD . . . 10

6.4 GloVe (Global Vectors) . . . 10

7 Sequence Modeling: RNNs, LSTMs, GRUs & BPTT **10**

7.1 Vanilla Recurrent Neural Network (RNN) . . . 10

7.2 Long Short-Term Memory (LSTM) . . . 10

7.3 Gated Recurrent Unit (GRU) . . . 11

8 Encoder-Decoder & Attention Mechanisms **11**

8.1 The Encoder-Decoder Bottleneck . . . 11

8.2 Bahdanau Additive Attention . . . 11

8.3 Self-Attention & Scaled Dot-Product . . . 11

9 CNN Mechanics, Receptive Fields & Visual Interpretability 11

9.1 Spatial Dimension Equation . . . 11
9.2 Parameter Counting in CNNs . . . 11
9.3 Key Architectural Milestones . . . 12
9.4 Interpretability Methods . . . 12

10 Perceptron Geometry, Separability & Convergence Proofs 12

10.1 Perceptron Fundamentals . . . 12
10.2 Novikoff's Perceptron Convergence Theorem . . . 12

11 Master Numerical Formula & Problem Tracing Sheet 12

11.1 Single Perceptron Step (No Bias) . . . 12
11.2 Single Sigmoid Prediction . . . 13
11.3 Gradient Calculation Tracing (Backprop Chain Rule) . . . 13
11.4 Mean Squared Error (MSE) . . . 13
11.5 Matrix Properties . . . 13
11.6 Batch Normalization Step . . . 13

Here is the complete, consolidated Master Exam Revision Document containing all sections from both the 12-Week Course Summary/Case Studies and the Deep Theoretical/Mathematical Revision Guide. You can save this directly as a PDF (e.g., in your browser or Markdown-to-PDF viewer: press Ctrl + P or Cmd + P → Save as PDF).

1 Core Optimization Algorithms & Contours

1.1 The Error Landscape & Gradient Descent (GD)

- Error Surface: A multi-dimensional hypersurface where parameters θ = [w, b] form the domain and the vertical height represents the loss L(θ).
- Standard GD Update Rule:
$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$
- Taylor Series Derivation of Descent Direction: For a small step u of length η ($\Delta\ \theta = \eta$ u where $\|u\| = 1$):
$$L(\theta + \eta u) \approx L(\theta) + \eta\, u^T \nabla_\theta L(\theta)$$
To ensure decrease in loss:
$$L(\theta + \eta u) - L(\theta) < 0 \implies \eta u^T \nabla_\theta L(\theta) < 0.$$
The inner product is minimized when u is pointing in the exact opposite direction of the gradient:
$$u = -\frac{\nabla_\theta L(\theta)}{\|\nabla_\theta L(\theta)\|}$$
- Contours Interpretation:
  - Closely spaced contour lines indicate high slope/curvature (large gradient magnitude $\implies$ large steps).
  - Widely spaced contour lines indicate gentle slopes or plateaus (small gradient magnitude $\implies$ updates crawl).
  - In elongated ravines/valleys, GD suffers from cross-valley oscillations while making negligible progress along the base.

1.2 Momentum-Based Gradient Descent

- Motivation: Overcome slow crawling on plateaus and dampen oscillations in ill-conditioned valleys.
- Update Equations:
$$v_t = \gamma v_{t-1} + \eta \nabla L(w_t) \quad \text{or} \quad v_t = \beta v_{t-1} + \eta g_t$$
$w_{t+1} = w_t$ - v_t (Note: If defined as $v_t = \gamma\ v_{t-1} + \nabla$ L(w_t), the update is $w_{t+1} = w_t$ - $\eta\ v_t$. Always align with the prompt's parameter placement.)
- Physical Intuition: A heavy ball rolling down an error landscape. Velocity v_t builds up speed along persistent gradient directions and cancels out orthogonal components that oscillate in sign.
- Limitation: Can severely overshoot the global/local basin minimum due to high momentum accumulation, resulting in large oscillations around the minimum.

1.3 Nesterov Accelerated Gradient (NAG)

- Philosophy: "Look before you leap."
- Update Equations: $w_{\text{lookahead}} = w_t$ - $\gamma\ v_{t-1}$
$$v_t = \gamma v_{t-1} + \eta \nabla L(w_{\text{lookahead}})$$
$$w_{t+1} = w_t - v_t$$
- Mechanism:
  1. Makes an interim step purely along the accumulated momentum vector to $w_{\text{lookahead}}$.
  2. Measures the gradient at that future point.
  3. If the momentum was about to overshoot and climb an opposing hill, the gradient at the look-ahead point points backward, applying an adaptive brake.

1.4 AdaGrad (Adaptive Gradient Algorithm)

- Motivation: Address feature sparsity (rare features receive tiny, infrequent updates in standard GD).
- Update Equations:

$$r_t = r_{t-1} + g_t \odot g_t \quad \text{(accumulated squared gradients)}$$
$$w_{t+1} = w_t - \frac{\eta}{\sqrt{r_t} + \epsilon} \odot g_t$$

- Pros: Features with small historical gradients have small r_t, boosting their effective learning rate.
- Cons (Fatal Flaw): Since $g_t^2 \geq 0$, r_t monotonically increases. The denominator continually blows up, causing the effective step size to shrink to zero and prematurely freezing learning.

1.5 RMSProp

- Motivation: Prevent the learning rate from monotonically vanishing in AdaGrad.
- Mechanism: Replaces the monotonic cumulative sum with an Exponentially Weighted Moving Average (EWMA): $v_t = \beta\ v_{t-1} + (1 - \beta)\ g_t^2$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t} + \epsilon} \odot g_t$$

- Behavior: Gives recent gradients higher weight than ancient history, allowing the effective learning rate to expand or contract based on current local surface geometry.

1.6 Adam (Adaptive Moment Estimation)

- Combines: First moment vector m_t (Momentum) + Second raw moment vector v_t (RMSProp).
- Equations:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\beta_1 \approx 0.9)$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\beta_2 \approx 0.999)$$

- Mathematical Derivation of Bias Correction:
  - Initial conditions: $m_0 = \mathbf{0}$, $v_0 = \mathbf{0}$.
  - Unrolling m_t recursively:

$$m_t = (1 - \beta_1) \sum_{i=1}^{t} \beta_1^{t-i} g_i$$

  - Taking expectations under the assumption that g_i comes from a stationary distribution ($\mathbb{E}[g_i] = \mathbb{E}[g]$):

$$\mathbb{E}[m_t] = \mathbb{E}\left[(1 - \beta_1) \sum_{i=1}^{t} \beta_1^{t-i} g_i\right] = \mathbb{E}[g](1 - \beta_1) \sum_{k=0}^{t-1} \beta_1^k = \mathbb{E}[g](1 - \beta_1) \left(\frac{1 - \beta_1^t}{1 - \beta_1}\right) = \mathbb{E}[g](1 - \beta_1^t)$$

  - Because of zero-initialization, m_t is biased towards zero by a factor of (1 - β_1^t).
  - Unbiased Estimators:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

- Final Parameter Update:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

2 Weight Initialization, Saturation & Batch Normalization

2.1 Pathologies of Naive Initialization

- Constant / All-Zero Initialization:
  - For any layer with z = Wx + b, if all w_{ij} are identical constants, every neuron receives identical inputs, computes identical activations, and receives identical error gradients during backpropagation:

$$\frac{\partial L}{\partial w_{1j}} = \frac{\partial L}{\partial w_{2j}} = \cdots = \frac{\partial L}{\partial w_{mj}}$$

  - Neurons never differentiate; this is known as the Symmetry Breaking Problem. The network collapses to having the representational capacity of a network with only 1 neuron per layer.
- Excessively Small Weights (Var(w) $\ll$ 1/n):
  - Activations contract exponentially across depth: $\text{Var}(h^{(l)}) \ll \text{Var}(h^{(l-1)})$.
  - Input signals vanish into the noise floor near 0, causing downstream gradients to extinguish.
- Excessively Large Weights (Var(w) $\gg$ 1/n):
  - Activations grow exponentially, driving non-linearities (sigmoid, tanh) deep into their asymptotic flat tails.
  - Local derivatives σ'(z) $\to$ 0, causing immediate gradient vanishing in early and middle layers.

2.2 Principled Initialization Schemes

- Variance Conservation Goal: Ensure signal variance neither explodes nor vanishes: $\text{Var}(a^{(l)}) = \text{Var}(a^{(l-1)})$
- Consider a linear aggregation z = $\sum_{i=1}^{n_{\text{in}}} w_i\, x_i$. Assuming w_i, x_i are independent with mean zero:

$$\text{Var}(z) = \sum_{i=1}^{n_{\text{in}}} \text{Var}(w_i x_i) = n_{\text{in}} \text{Var}(w) \text{Var}(x)$$

  For Var(z) = Var(x), we require:

$$\text{Var}(w) = \frac{1}{n_{\text{in}}}$$

- Xavier / Glorot Initialization (For Tanh, Sigmoid):
  - Assumes activations operate in linear regions around 0 (slope

$$\approx 1).$$

  - Forward Pass condition: Var(w) = $\frac{1}{n_{\text{in}}}$.
  - Backward Pass condition: Var(w) = $\frac{1}{n_{\text{out}}}$.
  - Harmonic Mean Compromise:

$$\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}} \quad \text{or uniform} \quad \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, +\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

- He / Kaiming Initialization (For ReLU, Leaky ReLU):
  - ReLU zeros out all negative pre-activations, eliminating approximately half the signal: $\mathbb{E}[(\max(0, z))^2]$ = $\frac{1}{2}$ Var(z)
  - Halving activation variance requires doubling the weight variance:

$$\text{Var}(w) = \frac{2}{n_{\text{in}}}$$

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

2.3 Activation Functions Comparison

Activation	Formula	Gradient $\frac{df}{dx}$	Output Range	Critical Vulnerability / Pros
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	$\sigma(z)(1-\sigma(z))$	$(0, 1)$	Saturation at both extremes; non-zero centered (causes zig-zag updates).
Tanh	$\frac{e^z-e^{-z}}{e^z+e^{-z}}$	$1-(\tanh(z))^2$	$(-1, 1)$	Zero-centered (solves zig-zag); still saturates at extremes.
ReLU	$\max(0, z)$	1 if $z > 0$ else 0	$[0, \infty)$	Non-saturating in positive domain; computationally fast; **Dying ReLU** if $z < 0$.
Leaky ReLU	$\max(\alpha z, z), \alpha \approx 0.01$	1 if $z > 0$ else α	$(-\infty, \infty)$	Eliminates Dying ReLU by providing non-zero gradient for negative inputs.

2.4 Batch Normalization (BN)

- Core Problem Addressed: Internal Covariate Shift — changes in the distribution of internal layer inputs during training caused by parameter updates in prior layers.
- Mini-Batch Transformation (applied to pre-activations x $\in \mathcal{B}$):
  1. Mini-Batch Mean: $\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i$
  2. Mini-Batch Variance:
  $$\sigma_{\mathcal{B}}^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2$$
  3. Normalization:
  $$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$
  4. Scale & Shift (Learnable Parameters γ, β):
  $$y_i = \gamma\hat{x}_i + \beta$$
- Why γ and β are critical: They preserve the network's capacity. If identity mapping is optimal, the network can learn $\gamma = \sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}$ and $\beta = \mu_{\mathcal{B}}$, fully undoing normalization.
- Inference / Test Mode: No mini-batches exist at test time. The layer uses accumulated population running averages:
$$\mu_{\text{run}} \leftarrow (1-\alpha)\mu_{\text{run}} + \alpha\mu_{\mathcal{B}}$$
$$\sigma_{\text{run}}^2 \leftarrow (1-\alpha)\sigma_{\text{run}}^2 + \alpha\sigma_{\mathcal{B}}^2$$
$$\hat{x}_{\text{test}} = \frac{x - \mu_{\text{run}}}{\sqrt{\sigma_{\text{run}}^2 + \epsilon}}$$

3 Linear Algebra Foundations: EVD, SVD & PCA

3.1 Eigenvalue Decomposition (EVD)

- Applies to: Square matrices only (n × n).
- Definition:
$$Au_i = \lambda_i u_i \implies AU = U\Lambda \implies A = U\Lambda U^{-1}$$
- Symmetric Matrix Theorem: If A = A^T, its eigenvalues are real, and its eigenvectors form an orthonormal set ($U^{-1} = U^T$):
$$A = U\Lambda U^T = \sum_{i=1}^{n} \lambda_i u_i u_i^T$$
- Quadratic Form Optimization: For unit vector $\|x\|_2 = 1$:
$$\max_{\|x\|=1} x^T Ax = \lambda_{\max}(A), \quad \arg\max_{\|x\|=1} x^T Ax = u_{\max}$$

3.2 Singular Value Decomposition (SVD)

- Applies to: Any real matrix A $\in \mathbb{R}^{m \times n}$.
- Factorization: A = U Σ V^T
  - U $\in \mathbb{R}^{m \times m}$: Columns are orthonormal eigenvectors of A A^T (Left singular vectors).
  - V $\in \mathbb{R}^{n \times n}$: Columns are orthonormal eigenvectors of A^T A (Right singular vectors).
  - $\Sigma \in \mathbb{R}^{m \times n}$: Diagonal matrix of singular values $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_r \geq 0$, where

$$\sigma_i = \sqrt{\lambda_i(A^T A)}.$$

- Geometric Interpretation: Rotation (V^T) $\rightarrow$ Axis Scaling (Σ) $\rightarrow$ Rotation (U).
- Low-Rank Matrix Truncation (Eckart-Young-Mirsky Theorem): The closest rank-k matrix A_k to A under Frobenius norm is obtained by retaining only the first k singular values:

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T, \quad \min_{\text{rank}(B)=k} \|A - B\|_F^2 = \|A - A_k\|_F^2 = \sum_{j=k+1}^{\min(m,n)} \sigma_j^2$$

3.3 Principal Component Analysis (PCA)

- Setup: Zero-centered data matrix X $\in \mathbb{R}^{N \times d}$.

$$\text{Sample Covariance Matrix:} \quad \Sigma = \frac{1}{N} X^T X \in \mathbb{R}^{d \times d}$$

- The 3 Unified Formulations:
  1. Maximum Variance: Find projection direction p ($\|p\|$=1) maximizing variance $\frac{1}{N}(Xp)^T$(Xp) = p^T Σ p.

$$\nabla_p[p^T \Sigma p - \lambda(p^T p - 1)] = 0 \implies \Sigma p = \lambda p$$

  2. Minimum Reconstruction Error: Orthogonal projection minimizing $\sum \|x_i - \hat{x}_i\|^2$ drops the eigenvectors corresponding to the smallest eigenvalues of Σ.
  3. Decorrelation: Linear transformation $\hat{X}$ = X P produces diagonal covariance: $\Sigma_{\hat{X}} = P^T$ Σ P = Λ
- Variance Explained:

$$\text{Proportion of Variance Explained by } k \text{ components} = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{j=1}^{d} \lambda_j}$$

4 Autoencoders, Weight Tying & Regularization

4.1 Autoencoder Formulations

- Encoder: Maps input to latent bottleneck: h = g(W_e x + b_e)
- Decoder: Reconstructs original input from latent code:

$$\hat{x} = f(W_d h + b_d)$$

- Loss Functions:
  - Real-valued: Squared error $\|x - \hat{x}\|^2$ (Linear activation on decoder output).
  - Binary: Cross-Entropy -$\sum$ [x_i log $\hat{x}_i$ + (1 - x_i)log(1 - $\hat{x}_i$)] (Sigmoid activation on decoder output).

4.2 Linear Autoencoders $\iff$ PCA Equivalence

- Theorem: If an autoencoder uses linear encoder (h = W_e x), linear decoder ($\hat{x}$ = W_d h), and squared error loss on zero-centered data, the weights span the identical subspace as Principal Component Analysis.
- Optimization problem: $\min_{W_e, W_d} \|X - XW_eW_d\|_F^2$. By the Eckart-Young Theorem, optimal solution converges to the truncated SVD subspace of the covariance matrix.

4.3 Structural & Functional Regularizations

- Weight Tying ($W_d = W_e^T$):
  - Forces decoder weight matrix to be the transpose of the encoder weight matrix.
  - Halves the parameter count for weights; prevents overfitting; serves as a structural prior.
- Denoising Autoencoders (DAE):
  - Corrupt input: $\tilde{x} \sim$ q($\tilde{x}$|x) (via Gaussian noise or dropout/masking).
  - Objective: min L(x, f(g($\tilde{x}$))) (Loss is evaluated against the clean target x).
  - Compels the network to project off-manifold noisy samples back onto the true low-dimensional data manifold.
- Sparse Autoencoders:
  - Penalizes hidden unit average activations:

$$\hat{\rho}_j = \frac{1}{N}\sum_{i=1}^{N} h_j(x^{(i)}).$$

  - Penalty uses KL divergence to target activation $\rho \ll 1$:

$$L_{\text{total}} = L_{\text{recon}} + \beta \sum_j \text{KL}(\rho \,||\, \hat{\rho}_j)$$

- Contractive Autoencoders (CAE):
  - Penalizes the squared Frobenius norm of the Jacobian of the encoder activations:

$$L_{\text{total}} = L_{\text{recon}} + \lambda \|J_h(x)\|_F^2 = L_{\text{recon}} + \lambda \sum_i \sum_j \left(\frac{\partial h_i}{\partial x_j}\right)^2$$

  - Forces the derivative $\frac{\partial h}{\partial x} \approx 0$ along directions orthogonal to the manifold.

5 Bias-Variance Tradeoff, True Error & Ensembling

5.1 Bias-Variance Decomposition

For target y = f(x) + ϵ with

$$\mathbb{E}[\epsilon] = 0, \text{Var}(\epsilon) = \sigma^2 :$$
$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(f(x) - \mathbb{E}[\hat{f}(x)])^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible Noise}}$$

- Underfitting: High bias, low variance.
- Overfitting: Low bias, high variance.

5.2 True Error vs. Empirical Training Error

- Training error is inherently optimistic: $\mathbb{E}$[Train Error] < $\mathbb{E}$[True Error]
- The gap is caused by the model fitting the specific noise realizations in the training set:

$$\text{Gap} = \frac{2}{N}\sum_{i=1}^{N} \text{Cov}(\hat{y}_i, y_i)$$

- Via Stein's Lemma, for Gaussian noise:

$$\text{Cov}(\hat{y}_i, y_i) = \sigma^2 \frac{\partial \hat{f}(x_i)}{\partial y_i}$$

  - A complex model has high sensitivity $\frac{\partial \hat{f}(x_i)}{\partial y_i}$, blowing up the gap between train and test performance.

5.3 Regularization Connections & Equivalences

- L2 Regularization via Hessian: Transform of weights:

$$\tilde{w} = (H + \alpha I)^{-1} H w^* = Q(\Lambda + \alpha I)^{-1} \Lambda Q^T w^*.$$

  - Direction with eigenvalue λ_i: Shrinkage factor is $\frac{\lambda_i}{\lambda_i + \alpha}$. High curvature preserved; low curvature (noise) eliminated.

- Input Noise ≡ L2 Regularization:

$$\mathbb{E}_\epsilon \left[(y - w^T(x + \epsilon))^2 \right] = (y - w^T x)^2 + \sigma^2 \|w\|^2$$

  Adding zero-mean Gaussian noise to inputs is mathematically equivalent to explicit L2 weight decay.

- Early Stopping ≈ L2 Regularization: In gradient descent on a quadratic bowl: w_t = [I - $(I - \eta H)^t$] w^* Comparing to L2's $\tilde{w}$ = [I - $(I + \alpha^{-1} H)^{-1}$] w^* demonstrates that t · $\eta \approx \frac{1}{\alpha}$. Early stopping restricts parameter movement to a bounded Euclidean radius, acting as an implicit L2 penalty.

5.4 Ensemble Error Formula & Dropout

- For k estimators with individual variance V and pairwise error covariance C:

$$\mathbb{E}[\text{Ensemble Error}] = \frac{1}{k} V + \left(1 - \frac{1}{k}\right) C$$

  - When predictions are uncorrelated (C = 0), variance decreases as $\frac{1}{k}$.

- Dropout as Ensemble Approximation:
  - During training: Every forward pass samples a thinned sub-network from $2^{|N|}$ possible configurations.
  - During inference: The full network is evaluated, scaling outgoing weights by keep probability p (or scaling training activations by 1/p via Inverted Dropout).
  - Prevents feature co-adaptation and approximates geometric averaging of an exponential number of subnetworks.

6 Word Representations, Word2Vec, GloVe & SVD Equivalence

6.1 Vector Representations

- One-Hot Encoding: Dimension |V|. Orthogonal (u^T v = 0 for all words). No semantic similarity; memory inefficient.

- Pointwise Mutual Information (PMI):

$$\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w)P(c)} = \log \left(\frac{\text{count}(w, c) \cdot N}{\text{count}(w) \cdot \text{count}(c)} \right)$$
$$\text{PPMI}(w, c) = \max(0, \text{PMI}(w, c))$$

6.2 Word2Vec Framework

- Continuous Bag-of-Words (CBOW): Averages context vectors to predict target center word: h = $\frac{1}{2C}$ $\sum_{-C \le j \le C, j \neq 0} v_{w_{t+j}}$

- Skip-Gram: Takes center word w_t to predict surrounding context words in window [-C, C].

- Negative Sampling (SGNS): Replaces computationally prohibitive |V|-way softmax denominator with k binary logistic classifications:

$$\mathcal{L} = \log \sigma(u_w^T v_c) + \sum_{i=1}^{k} \mathbb{E}_{v_{n_i} \sim P_n(w)} \left[\log \sigma(-u_w^T v_{n_i}) \right]$$

  - Negative samples drawn from unigram distribution smoothed to 3/4 power: P_n(w) ∝ $U(w)^{3/4}$.

6.3 The Grand Connection: Word2Vec $\iff$ SVD

- Levy & Goldberg Discovery: Word2Vec's Skip-gram with Negative Sampling (SGNS) is implicitly factorizing a shifted PMI matrix: M = PMI(w, c) - log k where k is the number of negative samples.

6.4 GloVe (Global Vectors)

- Combines global count matrices with log-bilinear dot-product optimization:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

where

$$f(X_{ij}) = \min \left(1, \left(\frac{X_{ij}}{x_{\max}} \right)^{\alpha} \right)$$

caps the influence of high-frequency words.

7 Sequence Modeling: RNNs, LSTMs, GRUs & BPTT

7.1 Vanilla Recurrent Neural Network (RNN)

- Forward Equations: $s_t = \tanh(\text{U } x_t + \text{W } s_{t-1} + \text{b})$

$$\hat{y}_t = \text{softmax}(V s_t + c)$$

- Backpropagation Through Time (BPTT):

$$\frac{\partial L_t}{\partial W} = \sum_{k=1}^{t} \frac{\partial L_t}{\partial s_t} \frac{\partial s_t}{\partial s_k} \frac{\partial s_k}{\partial W}$$

- Gradient Multiplications across Timesteps:

$$\frac{\partial s_t}{\partial s_k} = \prod_{j=k+1}^{t} \frac{\partial s_j}{\partial s_{j-1}} = \prod_{j=k+1}^{t} \text{diag}(1 - s_j^2) W^T$$

  - Exploding Gradients: Norm of product > 1 (Managed using Gradient Clipping: $\hat{g} = \frac{\theta}{\|g\|}$ g if $\|g\| > \theta$).
  - Vanishing Gradients: Norm of product < 1 (Prevents learning long-range dependencies).

7.2 Long Short-Term Memory (LSTM)

- Full Equations:

$$\begin{gathered}
\text{Forget Gate:} \quad f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f) \\
\text{Input Gate:} \quad i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i) \\
\text{Candidate Cell State:} \quad \tilde{c}_t = \tanh(W_c h_{t-1} + U_c x_t + b_c) \\
\textbf{Cell State Update}: \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\
\text{Output Gate:} \quad o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o) \\
\textbf{Hidden State}: \quad h_t = o_t \odot \tanh(c_t)
\end{gathered}$$

- Why LSTMs Solve Vanishing Gradients:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t$$

  - The gradient backpropagates through the cell state conveyor belt via addition.
  - If $f_t = 1$, the gradient propagates backward through time with zero attenuation.

7.3 Gated Recurrent Unit (GRU)

- Merges cell and hidden state into a single representation h_t; has only 2 gates: Reset Gate: $r_t = \sigma(W_r\ h_{t-1} + U_r\ x_t)$

$$\begin{aligned} \text{Update Gate:} \quad & z_t = \sigma(W_z h_{t-1} + U_z x_t) \\ \text{Candidate Hidden State:} \quad & \tilde{h}_t = \tanh(W(r_t \odot h_{t-1}) + U x_t) \\ \textbf{Final Hidden State}: \quad & h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \end{aligned}$$

8 Encoder-Decoder & Attention Mechanisms

8.1 The Encoder-Decoder Bottleneck

- Conventional Seq2Seq compresses an entire sequence into a single fixed-length context vector $\mathrm{c} = h_{T_{\text{in}}}$, causing performance to degrade as input length increases.

8.2 Bahdanau Additive Attention

- Alignment Score: $e_{ij} = v_a^T\ \tanh(W_a\ s_{i-1} + U_a\ h_j)$
- Attention Distribution (Softmax):

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}$$

- Dynamic Step-Specific Context Vector: $c_i = \sum_{j=1}^{T_x} \alpha_{ij}\ h_j$

8.3 Self-Attention & Scaled Dot-Product

- Linear projections to Queries (Q), Keys (K), Values (V):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Scaling Factor $\sqrt{d_k}$: If elements of q and k are independent variables with mean 0 and variance 1, their dot product has mean 0 and variance d_k. For large dimensions, large dot products push the softmax function into regions with near-zero gradients. Dividing by $\sqrt{d_k}$ normalizes the variance to 1.

9 CNN Mechanics, Receptive Fields & Visual Interpretability

9.1 Spatial Dimension Equation

Given input size W × H, square filter size F, padding P, stride S:

$$\text{Output Width } W_{\text{out}} = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1$$

9.2 Parameter Counting in CNNs

For a convolutional layer with K filters of size F × F, receiving input with C channels: Weight Parameters = K × (F × F × C)

$$\begin{aligned} \text{Biases} &= K \quad \text{(exactly 1 bias per filter)} \\ \text{Total Parameters} &= K \times (F \cdot F \cdot C + 1) \end{aligned}$$

9.3 Key Architectural Milestones

- VGGNet: Stacked small 3×3 filters. (Two stacked 3×3 filters yield an effective receptive field of 5×5 with fewer parameters:
$$2 \times (3^2) = 18 vs. 5^2 = 25).$$
- GoogLeNet: Inception module processing 1×1, 3×3, 5×5 convolutions in parallel; uses 1×1 convs as depth-reduction bottlenecks.
- ResNet: Residual skip connections solving the degradation problem:
$$H(x) = F(x) + x \implies \frac{\partial H}{\partial x} = \frac{\partial F}{\partial x} + 1$$
The +1 term ensures clean gradient flow directly through very deep networks.

9.4 Interpretability Methods

- Occlusion Sensitivity: Systematically occludes portions of the input image and evaluates drops in probability to localize decision-driving features.
- Saliency Maps: Gradients of the output score with respect to input pixels ($\nabla_x \ S_c(\mathrm{x})$).
- Guided Backpropagation: Sets negative gradients to zero during backpropagation through ReLU layers, suppressing noisy backwards signals.
- Gram Matrix (Neural Style Transfer): $G^l_{ij} = \sum_k \ F^l_{ik} \ F^l_{jk} = (F^l)(F^l)^T$ Measures feature channel correlations independent of spatial positioning to represent artistic style.

10 Perceptron Geometry, Separability & Convergence Proofs

10.1 Perceptron Fundamentals

- Decision boundary: Hyperplane $\sum_{i=1}^{d} \ w_i \ x_i + \mathrm{b} = 0$.
- If data is linearly separable, a single-layer perceptron converges to zero training classification error.
- XOR Non-Separability: Output cannot be separated by a single linear hyperplane because (0, 1) and (1, 0) are diagonally opposite to (0, 0) and (1, 1).

10.2 Novikoff's Perceptron Convergence Theorem

- Let inputs be augmented such that $\|x\| \leq \mathrm{R}$. Let an optimal separating unit hyperplane w^* exist with margin $\gamma > 0$ such that: $y_k \ (w^{*T} \ x_k) \geq \gamma \quad \forall \ \mathrm{k}$
- Perceptron update rule for mistake: $w_{k+1} = w_k + y_k \ x_k$ (starting with
$$w_0 = 0).$$
- Maximum Number of Updates M: $\mathrm{M} \leq \frac{R^2}{\gamma^2}$
- Guarantees convergence in a finite number of steps.

11 Master Numerical Formula & Problem Tracing Sheet

11.1 Single Perceptron Step (No Bias)

- Update condition: if $\mathrm{y}(w^T \ \mathrm{x}) \leq 0$:
$$w_{\text{new}} = w_{\text{old}} + yx \quad \text{where } y \in \{-1, +1\}$$

11.2 Single Sigmoid Prediction

$$z = \sum w_i x_i + b, \quad \hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Decision rule: $\hat{p} \geq 0.5 \iff$ z ≥ 0.

11.3 Gradient Calculation Tracing (Backprop Chain Rule)

For a single regression target using squared error loss:

$$L = \frac{1}{2}(\hat{y} - y)^2 \implies \frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

- If $\hat{y}$ = w x + b:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w} = (\hat{y} - y)x$$
$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b} = (\hat{y} - y)$$

- Gradient Descent Parameter Update:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

$b_{\text{new}} = b_{\text{old}}$ - η $\frac{\partial L}{\partial b}$ (Check: If $\hat{y} <$ y, gradient is negative $\implies$ w increases.)

11.4 Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

11.5 Matrix Properties

- Trace of a matrix: Sum of main diagonal elements = Sum of eigenvalues:

$$\text{Trace}(A) = \sum_i A_{ii} = \sum_i \lambda_i$$

- Determinant: Product of eigenvalues: det(A) = $\prod_i \lambda_i$

11.6 Batch Normalization Step

$$\mu = \frac{1}{m} \sum_{i=1}^{m} x_i, \quad \sigma^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu)^2$$
$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$